Continuous Clearing Auction Simulator

Agent-Based Model: Architecture and Mechanics

Kosmos Ventures

March 4, 2026

Abstract

We describe the design and implementation of an agent-based simulation of the Continuous Clearing Auction (CCA), a time-discretized uniform-price mechanism for onchain token sales. The simulator faithfully implements the mathematical model of the CCA—including deterministic supply schedules, proportional budget spreading, and uniform-price clearing via bisection—and populates it with six heterogeneous agent types whose behavioural rules encode the key economic forces identified in the theoretical analysis: the *exposure premium* of early entry (Proposition 1) versus the *informational option value* of delay (Proposition 2). A Monte Carlo and parameter-sweep infrastructure enables systematic exploration of how sentiment, demand intensity, information leakage, supply shape, and agent composition jointly determine clearing prices, surplus allocation, and participation dynamics.

Contents

1 Introduction and Motivation **3**

2 CCA Mechanism **3**
- 2.1 Time and Supply . . . 3
- 2.2 Bids and Spreading Rule . . . 4
- 2.3 Uniform-Price Clearing . . . 4
- 2.4 Surplus Computation . . . 4

3 Agent Taxonomy **5**
- 3.1 Common Agent State . . . 5
- 3.2 Market State Observable . . . 5
- 3.3 Default Belief Update . . . 5
- 3.4 Agent Type Specifications . . . 6
  - 3.4.1 EarlyBeliever . . . 6
  - 3.4.2 InformedTrader . . . 6
  - 3.4.3 MomentumTrader . . . 6
  - 3.4.4 OptionValueOptimizer . . . 7
  - 3.4.5 NoiseTrader . . . 7
  - 3.4.6 WhaleAgent . . . 7
- 3.5 Population Factory . . . 7

4 Information and Sentiment Model **8**
- 4.1 Public Signal Process . . . 8
- 4.2 Sentiment Dynamics . . . 8

5 Simulation Orchestration **9**
5.1 Single-Run Procedure . . . 9
5.2 Monte Carlo Runner . . . 9
5.3 Parameter Sweep . . . 10

6 Experimental Design **10**
6.1 Baseline Run . . . 10
6.2 Sentiment Sweep . . . 10
6.3 Demand Intensity Sweep . . . 10
6.4 Information Leakage Sweep . . . 10
6.5 Supply Schedule Comparison . . . 11
6.6 Agent Composition Sweep . . . 11
6.7 Extreme Scenarios . . . 11

7 Architecture Overview **11**

8 Mapping to Theory **12**

9 Implementation Notes **12**

1 Introduction and Motivation

A *Continuous Clearing Auction* (CCA) generalises the familiar single-shot uniform-price auction to a setting in which supply is released over a sequence of discrete periods and the market clears repeatedly. The mechanism is designed for onchain token sales and aims to eliminate the incentive to delay participation that plagues standard uniform-price formats.

The theoretical analysis establishes two competing forces:

(i) **Exposure premium.** Entering earlier expands the support of a bidder's budget schedule across more clearing periods, granting access to potentially high-surplus early clearings. When per-dollar surplus is front-loaded (weakly decreasing in time), early entry weakly dominates (Proposition 1).

(ii) **Informational option value.** When external information leakage refines beliefs about the fundamental value V or the future clearing-price path, delaying entry allows conditioning on improved signals. If the conditional surplus can change sign, optimal stopping with delayed entry strictly dominates unconditional commitment (Proposition 2).

The simulator instantiates both regimes through a heterogeneous population of agents and measures how the balance between these forces varies with market parameters.

This document describes:

- the CCA mechanism as implemented (Section 2);
- the agent taxonomy and behavioural rules (Section 3);
- the information and sentiment environment (Section 4);
- the simulation loop and Monte Carlo infrastructure (Section 5);
- the experimental design and parameter sweeps (Section 6).

2 CCA Mechanism

2.1 Time and Supply

Time is discrete, indexed by $k \in \{1, \ldots, T\}$. The auctioneer commits *ex ante* to a deterministic supply-release schedule: at each period k a token increment $q_k > 0$ is released, with total supply

$$Q \;=\; \sum_{k=1}^{T} q_k.$$

The schedule is constructed from a weight vector $\mathbf{w} = (w_1, \ldots, w_T)$ via normalisation:

$$q_k \;=\; Q \cdot \frac{w_k}{\sum_{j=1}^{T} w_j}. \tag{1}$$

Five parametric families are supported:

Table 1: Supply schedule types.

Type	Weight formula	Intuition
`uniform`	$w_k = 1$	Constant release
`front_loaded`	$w_k = \text{linspace}(2.0,\ 0.2,\ T)_k$	Heavy early release
`back_loaded`	$w_k = \text{linspace}(0.2,\ 2.0,\ T)_k$	Heavy late release
`bell_curve`	$w_k = \exp(-\frac{1}{2}x_k^2),\ \ x \in [-3, 3]$	Peak in middle
`custom`	User-provided vector	Arbitrary

2.2 Bids and Spreading Rule

A bidder submits a budget $B > 0$ (in the quote asset) and a maximum acceptable unit price (cap) $m \in (0, \infty]$ at an entry time τ. The mechanism maps this bid into a *budget schedule* $\big(b_k(\tau)\big)_{k=1}^{T}$ satisfying:

$$b_k(\tau) \geq 0, \qquad \sum_{k=\tau}^{T} b_k(\tau) = B, \qquad b_k(\tau) = 0 \ \text{ for } k < \tau. \tag{2}$$

The default (canonical) spreading rule is **proportional-to-remaining-supply**:

$$b_k(\tau) \ = \ B \cdot \frac{q_k}{\sum_{j=\tau}^{T} q_j}, \qquad k \geq \tau. \tag{3}$$

This allocates budget to each future period in proportion to that period's share of the remaining supply. Two alternative rules are also implemented for sensitivity analysis:

- **Equal-weight**: $b_k(\tau) = B/(T - \tau)$ for $k \geq \tau$.
- **Front-weighted** (geometric decay δ): $b_k(\tau) \propto \delta^{k-\tau}$, normalised to sum to B.

The spreading rule interface is pluggable: any function mapping a bid and supply schedule to a conforming budget vector can be substituted.

2.3 Uniform-Price Clearing

At each period k, the mechanism performs the following steps:

1. **Collect active fragments.** For every bid i with entry time $\tau_i \leq k$, the budget fragment is $b_k^{(i)} = b_k(\tau_i)$.
2. **Aggregate demand curve.** At candidate price p, bid i demands

   $$d_k^{(i)}(p) \ = \ \frac{b_k^{(i)}}{p} \, \mathbf{1}\{p \leq m_i\}$$

   tokens. Total demand is

   $$D_k(p) \ = \ \sum_i d_k^{(i)}(p) \ = \ \frac{1}{p} \sum_{i:\, m_i \geq p} b_k^{(i)}. \tag{4}$$

   Note that $D_k(\cdot)$ is monotone decreasing in p within each piece between consecutive caps.
3. **Find the clearing price.** The uniform clearing price P_k solves $D_k(P_k) = q_k$. Since D_k is monotone decreasing, this is found by **bisection** on the interval $[p_{\min}, p_{\max}]$ with tolerance 10^{-8} and a maximum of 200 iterations.
4. **Execute.** Each bid i with $P_k \leq m_i$ spends $b_k^{(i)}$ and receives $b_k^{(i)}/P_k$ tokens. Bids with $P_k > m_i$ do not transact; their fragment goes unspent.

Edge cases.

- If $D_k(p_{\min}) \leq q_k$ (total demand below supply even at the floor), set $P_k = p_{\min}$.
- If $D_k(p_{\max}) \geq q_k$ (excess demand at the ceiling), set $P_k = p_{\max}$.
- If no bids are active, P_k defaults to $p_{\min}$.

2.4 Surplus Computation

For each bid i with valuation $V^{(i)}$, the per-period realised surplus at period k is:

$$\pi_k^{(i)}(\tau, m) \ = \ \mathbf{1}\{P_k \leq m\} \left(\frac{V^{(i)}}{P_k} - 1 \right) b_k(\tau). \tag{5}$$

The term $V^{(i)}/P_k - 1$ is the surplus per dollar spent: spending b_k at price P_k buys b_k/P_k tokens, each worth $V^{(i)}$, yielding a dollar profit of $(V^{(i)} - P_k) \cdot b_k/P_k = (V^{(i)}/P_k - 1)\, b_k$.

Total surplus aggregates across all periods of participation:

$$\Pi^{(i)} \;=\; \sum_{k=\tau}^{T} \pi_k^{(i)}(\tau, m). \tag{6}$$

For each bid, the simulator records: per-period surplus array, total surplus, total tokens acquired, total spend, unspent budget, budget-weighted average execution price, and surplus per dollar spent.

3 Agent Taxonomy

The simulation populates the CCA with six agent types, each encoding a distinct behavioural regime. All agents share a common state and belief-update mechanism, but differ in entry logic, cap selection, and—in some cases—belief-formation overrides.

3.1 Common Agent State

Every agent i maintains:

- Budget $B^{(i)} > 0$ in the quote asset.
- Base valuation $V_{\text{base}}^{(i)}$ (prior fundamental value) and uncertainty σ_V.
- Running posterior: belief mean $\hat{\mu}$ and variance $\hat{\sigma}^2$.
- Current valuation $\hat{V}^{(i)} = \hat{\mu}$ (set after each belief update).
- Sentiment sensitivity $\gamma_i \in (0, \infty)$, controlling weight on public signals.
- Entry state: whether the agent has entered, and if so, the entry period τ_i.

3.2 Market State Observable

At each period k, every agent observes a `MarketState` object containing: the current period; the history of clearing prices $P_0, \ldots, P_{k-1}$; cumulative and remaining supply; the full supply schedule (q_k); the current public signal s_k; the aggregate sentiment index $\sigma_k \in [0, 1]$; and the realised log-price volatility.

3.3 Default Belief Update

Unless overridden by a specific agent type, every agent performs a *two-stage conjugate-normal update* each period.

Stage 1: Public signal update. Define the prior precision $\lambda_{\text{prior}} = 1/\hat{\sigma}^2$ and the signal precision $\lambda_{\text{sig}} = \gamma/\sigma_V^2$, where γ is the agent's sentiment sensitivity. Given public signal s_k:

$$\hat{\mu}_{\text{post}} = \frac{\lambda_{\text{prior}}\, \hat{\mu} + \lambda_{\text{sig}}\, s_k}{\lambda_{\text{prior}} + \lambda_{\text{sig}}}, \qquad \hat{\sigma}^2_{\text{post}} = \frac{1}{\lambda_{\text{prior}} + \lambda_{\text{sig}}}. \tag{7}$$

Stage 2: Price signal update. If a clearing price $P_{k-1} > 0$ is available, it is treated as an additional signal about V after scaling by a factor of $1.2\times$ (reflecting that the clearing price is an upward-biased estimator of fundamental value in high-demand settings). The signal precision is $\lambda_{\text{price}} = 0.5/\nu^2$, where ν is the realised log-price volatility. A second conjugate update is applied:

$$\hat{\mu}' = \frac{\lambda'_{\text{prior}}\, \hat{\mu}_{\text{post}} + \lambda_{\text{price}} \cdot 1.2 P_{k-1}}{\lambda'_{\text{prior}} + \lambda_{\text{price}}}, \qquad (\hat{\sigma}')^2 = \frac{1}{\lambda'_{\text{prior}} + \lambda_{\text{price}}}, \tag{8}$$

where $\lambda'_{\text{prior}} = 1/\hat{\sigma}^2_{\text{post}}$. After updating, the agent sets $\hat{V}^{(i)} = \hat{\mu}'$.

3.4 Agent Type Specifications

Each agent type implements three methods: `decide_entry`$(k, \text{state}) \to \text{bool}$, `decide_cap`$(k, \text{state}) \to \mathbb{R}_{>0}$, and optionally overrides `update_beliefs`.

3.4.1 EarlyBeliever

Concept. High-conviction agent that enters very early, corresponding to the regime where Proposition 1 dominates (front-loaded surplus expectation).

Parameters (sampled per agent):

- `conviction_multiplier` $\sim U(1.3,\ 1.8)$: scales valuation to set cap.
- `max_entry_delay` $\sim \{0, 1, 2\}$: latest period to enter.

Entry logic. At each period $k \leq$ `max_entry_delay`, enter with probability $1-(0.3-0.2\cdot\sigma_k)$, where σ_k is the sentiment index. Entry is forced at the maximum delay period.

Cap. $m = \hat{V} \cdot$ `conviction_multiplier`.

Beliefs. Default (Section 3.3).

3.4.2 InformedTrader

Concept. Receives private signals about V and conditions entry on signal quality. Models the option-value regime of Proposition 2.

Parameters:

- `signal_precision` $\sim U(1, 4)$: precision of private signal.
- `entry_threshold` $\sim U(0.05, 0.20)$: minimum expected surplus per dollar.
- `signal_delay` $\sim \{1, 2, 3, 4\}$: periods before private signal arrives.

Private signal model. At each period $k \geq$ `signal_delay`, the agent draws

$$s_k^{\text{priv}} = V_{\text{base}} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}\left(0,\ \frac{1}{\texttt{signal_precision}}\right),$$

and performs a Bayesian conjugate update incorporating this private signal on top of the standard two-stage update.

Entry logic. Wait until $k \geq$ `signal_delay`. Estimate expected surplus per dollar as $\hat{\mu}/\hat{P}-1$, where $\hat{P}$ is the last observed clearing price (or $0.8\,V_{\text{base}}$ as a prior). Enter if this exceeds a time-decaying threshold:

$$\frac{\hat{\mu}}{\hat{P}} - 1 \ > \ \texttt{entry_threshold} \cdot \frac{T-k}{T}.$$

Cap. $m = \hat{V} \cdot 1.1$ (tight cap).

3.4.3 MomentumTrader

Concept. FOMO-driven agent that enters on price trend confirmation.

Parameters:

- `lookback` $\sim \{2, 3, 4, 5\}$: window for momentum computation.
- `momentum_threshold` $\sim U(0.01, 0.05)$: minimum mean log-return.
- `fomo_factor` $\sim U(0.5, 2.0)$: scales entry probability.

Momentum. Computed as the mean of $\Delta \log P$ over the last `lookback` valid prices.

Entry logic.

- If *momentum* $>$ `threshold`, enter with probability $\min(1,\ \texttt{fomo_factor}\cdot \textit{momentum}/\texttt{threshold})$.
- *Late-stage panic*: if $k > 0.8\,T$, enter with probability proportional to urgency $= (k - 0.8T)/(0.2T)$, capped at 0.5.

Cap. $m = \max(\text{recent_price} \times 1.2,\ \hat{V})$.

Belief override. Strong price anchoring: $\hat{V} \leftarrow 0.6\,\hat{V} + 0.4\,P_{k-1} \cdot 1.1$.

3.4.4 OptionValueOptimizer

Concept. Explicitly models the exposure premium vs. option value tradeoff formalised in Proposition 2.

Parameters:

- `patience` $\sim U(0.3, 1.0)$: weight on the option-value term.
- `option_value_decay` $\sim U(0.75, 0.95)$: per-period decay.

Option value estimate. The option value captures the Proposition 2 intuition that the value of waiting is proportional to uncertainty times the square root of remaining time:

$$\mathrm{OV}(k) \;=\; \texttt{patience} \cdot \sqrt{\hat{\sigma}^2} \cdot \sqrt{T-k} \cdot \texttt{decay}^{\,k}. \tag{9}$$

Exposure premium estimate:

$$\mathrm{EP}(k) \;=\; \max\!\left(\frac{\hat{\mu}}{\hat{P}} - 1,\; 0\right) \cdot \frac{q_k}{\sum_{j \geq k} q_j} \cdot B. \tag{10}$$

Entry logic. Enter when $\mathrm{EP}(k) > \mathrm{OV}(k)$. If fewer than 15% of periods remain, enter whenever expected surplus is positive.

Cap. $m = \hat{V} \cdot 1.05$ (very tight—sophisticated agent).

3.4.5 NoiseTrader

Concept. Behavioural retail participation: random entry, noisy cap, provides liquidity.

Parameters:

- `entry_probability` $\sim U(0.08, 0.28)$: per-period base entry rate.
- `cap_noise_std` $\sim U(0.1, 0.5)$: log-normal noise on cap.

Entry logic. At each period, enter with probability `entry_probability` $\cdot\, (0.5 + \sigma_k)$.

Cap. $m = \hat{V} \cdot e^{\varepsilon}$, with $\varepsilon \sim \mathcal{N}(0, \texttt{cap_noise_std})$, floored at 0.01.

Belief override. Sticky beliefs: $\hat{V} \leftarrow 0.9\,\hat{V} + 0.1\,P_{k-1}$.

3.4.6 WhaleAgent

Concept. Large-budget agent with market-impact awareness that splits entry across multiple periods (violates the price-taking Assumption 1 of the theory).

Parameters:

- `split_periods` $\sim \{2, 3, 4\}$: number of tranches.
- `impact_awareness` $\sim U(0.5, 1.0)$: discounts cap to avoid self-impact.
- `entry_start` $\sim \{1, 2, 3\}$: earliest entry period.

Entry logic. Compute target periods spaced by `stride` $= (T - \texttt{entry_start})/\texttt{split_periods}$. At each target period, enter if the last clearing price is below $1.1 \times \hat{V}$. Each entry uses budget $B/\texttt{split_periods}$.

Important: Unlike other agents which submit a single bid, whales submit **multiple bids** (one per tranche). The simulation loop handles this explicitly.

Cap. $m = \hat{V} \cdot (1 - \texttt{impact_awareness} \cdot 0.1)$.

3.5 Population Factory

The function `create_agent_population` draws heterogeneous agents from the distributions specified above. Key parameters:

Valuation sampling. For each agent i:

$$V_i \;=\; V_{\text{base}} + \underbrace{(\texttt{sentiment} - 0.5) \cdot 2 \cdot \texttt{dispersion} \cdot V_{\text{base}}}_{\text{sentiment shift}} + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0,\; \texttt{dispersion} \cdot V_{\text{base}}), \tag{11}$$

Table 2: Population factory parameters.

Parameter	Default	Description
`base_valuation`	10.0	Centre of valuation distribution
`valuation_dispersion`	0.2	Std as fraction of base valuation
`budget_mean`	1000	Mean budget (quote asset)
`budget_std`	300	Budget standard deviation
`whale_budget_mult`	10.0	Whale budget = mean × multiplier
`sentiment`	0.5	Shifts the valuation distribution

floored at 0.1. This shifts the entire valuation distribution upward (downward) when sentiment exceeds (falls below) the neutral level of 0.5.

Budget sampling. Drawn from $\mathcal{N}(\texttt{budget_mean}, \texttt{budget_std})$, floored at 50. Whales receive $\texttt{budget_mean} \cdot \texttt{whale_mult} \cdot (1 + \mathcal{N}(0, 0.2))$.

4 Information and Sentiment Model

4.1 Public Signal Process

At each period k, a public signal about the true fundamental value V_{true} is released:

$$s_k \;=\; V_{\text{true}} + \varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}\Big(0,\ \big(0.5 \cdot V_{\text{true}} \cdot e^{-\lambda k}\big)^2\Big), \tag{12}$$

where λ is the *leakage rate*. Higher λ means faster convergence of public information to the true fundamental, modelling the filtration $\mathcal{F}_k$ from the theory becoming increasingly informative over time. At $\lambda = 0$ the signal noise remains constant; as $\lambda \to \infty$ the signal becomes exact immediately.

4.2 Sentiment Dynamics

Sentiment follows a bounded mean-reverting process with jumps:

$$\sigma_{k+1} \;=\; \alpha\, \sigma_k + (1 - \alpha)\, \bar{\sigma} + \varepsilon_k + J_k, \tag{13}$$

where:

- $\alpha = 0.9$ is the mean-reversion speed,
- $\bar{\sigma}$ is the long-run level (defaults to the initial sentiment),
- $\varepsilon_k \sim \mathcal{N}(0, \sigma_\varepsilon)$ is diffusive noise,
- $J_k = \pm\texttt{shock_magnitude}$ with probability `shock_probability`, else 0.

The process is clipped to $[0, 1]$ after each step.

Table 3: Information model parameters.

Parameter	Default	Description
`fundamental_value`	10.0	True V
`leakage_rate`	0.05	Per-period information leakage λ
`sentiment_mean_rev`	0.9	α
`sentiment_vol`	0.05	σ_ε
`shock_probability`	0.05	Probability of sentiment jump per period
`shock_magnitude`	0.3	Absolute size of jump

5 Simulation Orchestration

5.1 Single-Run Procedure

A single simulation run proceeds as follows:

Algorithm 1 Single CCA Simulation Run

Require: `SimulationConfig` with all parameters
Ensure: Result dict with clearing prices, agent results, diagnostics

1: Build supply schedule $\mathbf{q}$ from config ▷ Eq. (1)
2: Generate public signals $s_0, \ldots, s_{T-1}$ ▷ Eq. (12)
3: Generate sentiment path $\sigma_0, \ldots, \sigma_{T-1}$ ▷ Eq. (13)
4: Create agent population
5: `all_bids` $\leftarrow []$; `prices` $\leftarrow \mathbf{0}_T$; `entered` $\leftarrow \emptyset$
6: **for** $k = 0$ **to** $T - 1$ **do**
7: Compute realised volatility from `prices`$[0 : k]$
8: Build `MarketState` from current observables
9: **for** each agent i **do**
10: i.`update_beliefs`$(k, \texttt{state})$ ▷ Eqs. (7)–(8)
11: **if** i is WhaleAgent **then**
12: **if** i.`decide_entry`$(k, \texttt{state})$ **then**
13: Create bid with budget $= B_i/$`split_periods`
14: Append to `all_bids`
15: **end if**
16: **else if** $i \notin$ `entered` **then**
17: **if** i.`decide_entry`$(k, \texttt{state})$ **then**
18: Create bid with full budget B_i and cap i.`decide_cap`(k)
19: Append to `all_bids`; `entered` $\leftarrow$ `entered` $\cup \{i\}$
20: **end if**
21: **end if**
22: **end for**
23: Run ClearingEngine on `all_bids` with $\mathbf{q}$ ▷ Section 2.3
24: `prices`$[k] \leftarrow P_k$ from engine result
25: **end for**
26: Run final full auction on `all_bids` for definitive prices
27: Compute per-bid surplus decomposition ▷ Eq. (5)
28: **return** result

Remark 1 (Re-running the full auction each period)**.** *At step 16, the auction engine is called with* all *bids accumulated so far—not just new bids—because the spreading rule and clearing prices depend on the full bid set. New bids entering at period k change the demand curve and can shift budget allocations of earlier bids. The final run (step 18) with the complete bid set produces the definitive clearing prices and surplus computations. The complexity is $O(T \times N)$ per run where N is the total bid count.*

5.2 Monte Carlo Runner

The function `run_monte_carlo(config)` executes `config.n_runs` independent simulation runs, each with a deterministic seed `base_seed` $+ i$ for $i = 0, \ldots, n_{\text{runs}} - 1$. All randomness flows through `numpy.random.Generator` objects passed explicitly, avoiding global state.

5.3 Parameter Sweep

`parameter_sweep`(`base_config`, `param_name`, `param_values`, `n_runs_per`) iterates over each value in `param_values`, creates a modified configuration, runs n_{runs} Monte Carlo trials, and aggregates results into a `DataFrame` with columns:

Table 4: Sweep output columns.

Column	Description
`param_value`	Value of swept parameter for this run
`run_id`	Monte Carlo run index
`mean_clearing_price`	Mean of positive clearing prices
`final_clearing_price`	Last clearing price P_T
`price_volatility`	Std of positive clearing prices
`price_drift`	$(P_T/P_1) - 1$
`n_bids`	Total bids submitted
`total_surplus`	Sum of all agent surpluses
`surplus_{Type}`	Total surplus for each agent type
`avg_surplus_{Type}`	Average surplus per agent per type

6 Experimental Design

The simulator runs seven experiments, each targeting a specific dimension of the CCA's parameter space.

6.1 Baseline Run

A single run with $T = 24$, $Q = 5{,}000$, uniform supply, 71 agents (12 Early, 10 Informed, 15 Momentum, 6 OptionValue, 25 Noise, 3 Whale), $V = 10$, `sentiment` $= 0.55$, $\lambda = 0.05$. Produces a seven-panel diagnostic dashboard.

6.2 Sentiment Sweep

Sweeps `sentiment` $\in \{0.1, 0.2, \ldots, 0.9\}$ with 15 runs each.

Expected: Higher sentiment $\rightarrow$ higher clearing prices (more demand), higher participation, but also higher surplus for early and option-value agents since valuations shift upward while prices lag.

6.3 Demand Intensity Sweep

Sweeps `budget_mean` $\in \{300, 500, 750, 1{,}000, 1{,}500, 2{,}000, 3{,}000\}$ with 15 runs each.

Expected: Higher budgets $\rightarrow$ higher clearing prices (more dollars chasing the same tokens), increased volatility, and compression of surplus across agent types as competition intensifies.

6.4 Information Leakage Sweep

Sweeps `leakage_rate` $\in \{0.0, 0.02, 0.05, 0.1, 0.15, 0.25, 0.4\}$ with 15 runs each.

Expected: Higher leakage $\rightarrow$ InformedTraders lose edge (public signal converges to V faster); OptionValueOptimizers see diminished option value (less uncertainty to exploit). This experiment directly tests the Proposition 1 vs. Proposition 2 tradeoff.

6.5 Supply Schedule Comparison

Single runs for each of the four parametric schedule types: uniform, front-loaded, back-loaded, and bell-curve.

Expected: Front-loaded supply → higher initial prices (scarce later supply drives late demand); back-loaded → lower early prices. Bell-curve should exhibit volatility spikes at the peak-release period.

6.6 Agent Composition Sweep

Sweeps `n_informed` $\in \{0, 3, 6, 10, 15, 20, 30\}$ (other counts fixed) with 15 runs each.

Expected: More informed traders → more price discovery (prices converge to V), lower volatility, compression of NoiseTrader surplus.

6.7 Extreme Scenarios

Four named scenarios, single run each:

Table 5: Extreme scenario configurations.

Scenario	Key parameters
Euphoria	$\sigma_0 = 0.9$, budget= 2000, $\lambda = 0.01$, $\sigma_\varepsilon = 0.02$
Panic	$\sigma_0 = 0.15$, budget= 600, $\lambda = 0.2$, $\sigma_\varepsilon = 0.12$, shock prob = 0.15
Whale-dominated	8 whales, 3 Early, 5 Noise, 3 Momentum, whale mult = 20
Retail-heavy	60 Noise, 5 Early, 2 Informed, 5 Momentum, 1 OptVal, 0 Whale

7 Architecture Overview

The codebase is organised into five modules:

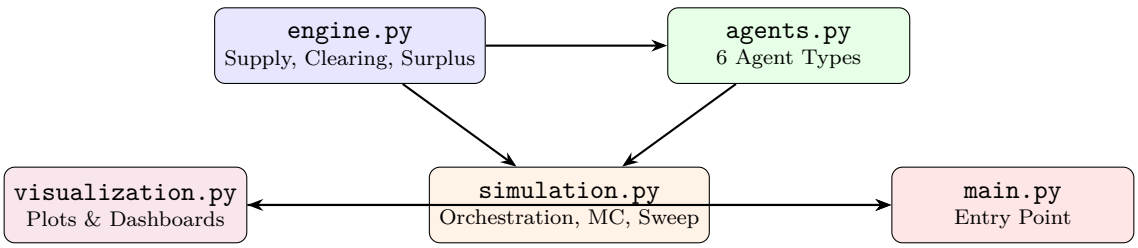

Figure 1: Module dependency graph.

- **`engine.py`** — CCA mechanism primitives: supply schedule construction, three spreading rules, the bisection-based clearing solver, and surplus decomposition.
- **`agents.py`** — The six agent types (Section 3.4), the abstract `BaseAgent` with the conjugate belief-update protocol, the `MarketState` observable, and the population factory.
- **`simulation.py`** — The information model (public signals, sentiment dynamics), the single-run orchestrator (Algorithm 1), the Monte Carlo runner, and the parameter-sweep infrastructure.
- **`visualization.py`** — All plotting routines: the 7-panel single-run dashboard, 4-panel parameter-sweep charts, and 3-panel scenario comparison.
- **`main.py`** — Entry point that executes all seven experiments and writes outputs (PNG figures and CSV data files).

8 Mapping to Theory

Table 6 summarises how each theoretical primitive maps to the simulation.

Table 6: Correspondence between theoretical model and simulation components.

Theory	**Simulation**	**Reference**
Discrete time $k \in \{1, \ldots, T\}$	Period loop in Algorithm 1	Section 5.1
Supply increment q_k	`build_supply_schedule`	Section 2.1
Spreading rule $b_k(\tau)$	`proportional_spreading`	Section 2.2, Eq. (3)
Clearing price P_k s.t. $D(P_k) = q_k$	`find_clearing_price` (bisection)	Section 2.3
Surplus $\pi_k^{(i)}$	`compute_surplus`	Section 2.4, Eq. (5)
Prop. 1 (front-loaded surplus $\Rightarrow$ early entry)	EarlyBeliever behaviour	Section 3.4.1
Prop. 2 (option value of waiting)	InformedTrader, OptionValueOptimizer	Sections 3.4.2, 3.4.4
Assumption 1 (price-taking)	All agents except WhaleAgent	Section 3.4.6
Assumption 3 (information arrival)	Private signals, public signal leakage	Sections 3.4.2, 4.1
Filtration $\mathcal{F}_k$	`MarketState` + belief updates	Sections 3.2, 3.3
Conditional surplus $g_k(\tau, m)$	Emergent from clearing + belief dynamics	Sections 2.3, 2.4

9 Implementation Notes

Numerical precision. The clearing-price bisection converges to 10^{-8} relative tolerance in at most 200 iterations. The demand function (4) is piecewise continuous with discontinuities at each agent's cap; bisection handles this correctly without special treatment.

RNG discipline. All randomness flows through `numpy.random.Generator` objects instantiated with explicit seeds. Each Monte Carlo run receives seed `base_seed` + `run_id`, and each agent receives its own child generator to avoid cross-contamination.

Computational cost. Each single run is $O(T \times N)$, where N is the total number of bids. The period-by-period loop re-runs the full auction at each step to correctly account for newly entering bids—this is intentional and faithful to the mechanism. For the default configuration ($T = 24$, $N \approx 70$), a single run completes in under one second; the full experiment suite (including all parameter sweeps with 15 runs each) executes in approximately 90 seconds.

Extensibility. The spreading rule is a pluggable function. New agent types can be added by subclassing `BaseAgent` and implementing `decide_entry` and `decide_cap`. Supply schedules accept arbitrary weight vectors via the `custom` type.